

Bike Sharing Demand MLOps Pipeline

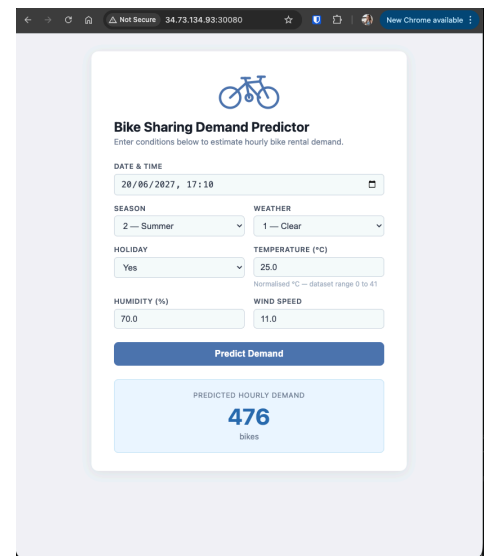
Repository Link: <https://github.com/caglazezgen/bike-sharing-mlops-pipeline>

1. Introduction

This report documents the MLOps architecture for a bike-sharing demand forecasting system. The pipeline spans data acquisition, preprocessing, training/testing, versioning, continuous training and monitoring, and automated deployment to a Kubernetes environment. The service is a FastAPI application packaged in Docker and deployed to a Kind cluster on a GCP VM. The architecture and workflow design follow established MLOps and software delivery practices with reproducibility, traceability, and safe deployment as primary goals.

2. Artifact Overview

- Dataset: Kaggle Bike Sharing Demand
- Target: hourly bike rental count
- Model: XGBoost regressor with log1p target transform
- Serving: FastAPI with /health and /ready for probes and /predict for inference
- Tracking: MLflow artifacts and metrics
- Versioning: semantic model versions stored in artifacts/model_versions.json



3. Branching Strategy

GitFlow-lite is used:

- main: production-ready releases (deployment source)
- develop: integration branch
- feature/*: new features and experiments
- release/*: release stabilization
- hotfix/*: production fixes

4. MLOps Architecture

4.1 System Components

1. Data layer: Kaggle ingestion produces raw data, then preprocessing generates processed datasets, feature configuration, and data profile artifacts. DVC tracks raw/bronze data pointers in Git, while the payloads are stored in GCS for reproducibility and auditability.

2. Training layer: The training pipeline executes XGBoost training and produces model artifacts, metrics, and metadata. Model versioning is semantic (major.minor.patch) with full provenance in artifacts/model_versions.json.

3. Serving layer: FastAPI provides UI and prediction endpoints. /health (liveness) and /ready (readiness) distinguish process health from model availability.

4. Deployment layer: GitHub Actions builds the container, pushes to Artifact Registry, and deploys to Kind on GCP VM. Rollout verification and smoke checks provide operational safety gates.

5. Monitoring layer: Drift detection runs on scheduled and workflow events using PSI and KS/Chi2 tests. If drift is detected beyond thresholds, continuous training is triggered.



4.2 Pipeline and Workflow Orchestration

- **Ingest and Preprocess:** downloads data, validates schema and ranges, writes dataset profiles, and tracks raw data via DVC.
- **Train Model:** trains the model, compares challenger vs champion, and uploads model artifacts.
- **Continuous Training:** retrains when bronze data changes; validates data quality before training.
- **Continuous Monitoring:** compares reference data to new data and triggers CT if drift exceeds thresholds.
- **Deploy:** builds and pushes Docker image, deploys to Kubernetes, verifies rollout, and performs smoke tests.

4.3 Reliability and Safety Controls

- **Readiness vs liveness probes** ensure pods do not serve traffic without a model loaded.
- **Immutable image tags** using commit SHA improve traceability and rollback safety.
- **Quality gate** blocks deployment if RMSLE regresses beyond the tolerance threshold.
- **Rollback logic** is activated when rollout checks fail (when a previous revision exists).

4.4 Explainability and Auditability

- **SHAP feature importance** is logged as an artifact for interpretability.
- **Model version history** includes metrics, hyperparameters, and dataset fingerprints.
- **Dataset metadata** captures raw and processed hashes for lineage.

4.5 Sustainability

- Training logs include emissions estimates (CodeCarbon), enabling reporting of compute impact.
- Short training times and data reuse through DVC minimize redundant compute.

5. Test Use Cases

- API prediction endpoint returns predictions for valid input.
- /health and /ready endpoints verify liveness and readiness semantics.
- Feature engineering adds required time features.
- Data validation rejects malformed timestamps and schema violations.

6. MLOps Stages

6.1 Business Problem Understanding

Bike-sharing demand forecasting is a time-series regression problem aimed at predicting hourly rentals to support bike rebalancing and operational planning. The target (count) reflects total rentals (casual + registered), and the design focuses on error sensitivity in low-demand vs high-demand periods. RMSLE was selected because it penalises under-prediction proportionally, aligning with operational risk when demand spikes.

6.2 Data Acquisition

Data is acquired via the Kaggle API using secured credentials. The ingest workflow downloads the competition dataset, validates the schema and basic integrity, and tracks the raw files with DVC. This ensures reproducibility, auditability, and low repository bloat while storing data payloads in GCS.

6.3 ML Methodology

The model is an XGBoost regressor trained on engineered temporal and weather features. The target is log1p-transformed to reduce skew and align with RMSLE optimisation. Feature selection removes redundant and leakage-prone inputs (e.g., `casual`, `registered`, `atemp`, `workingday`). Outliers are handled through Winsorization to stabilise gradients.

6.4 ML Training and Testing

Training runs in GitHub Actions using reproducible preprocessing and a fixed configuration file. Metrics logged per run include RMSLE, MAE, MAPE, and R². A champion vs challenger quality gate blocks deployments when RMSLE regresses beyond the 5% tolerance. Unit tests cover ingestion, preprocessing, API behaviour, and drift utilities.

6.5 Continuous Integration

CI runs on pull requests to `main` and `develop`. It installs dependencies, performs lint checks, and executes pytest with coverage thresholds. This prevents integration of failing tests or regressions into the shared branches.

6.6 Continuous Delivery

Successful training on `main` triggers the Deploy workflow. The pipeline builds a Docker image, pushes to Artifact Registry, applies Kubernetes manifests, verifies rollout status, and executes a smoke test against the /ready endpoint. Rollback logic is included for failed rollouts when a previous revision exists.

6.7 Continuous Training

Continuous Training (CT) triggers on changes to `data/bronze/train.csv.dvc`. New bronze data is validated, preprocessed, and used for retraining. Model metrics are compared to the champion model to prevent degradation, and artifacts are re-published for downstream deployment.

6.8 Continuous Monitoring

Monitoring compares new data with a reference dataset using PSI and KS/Chi2 tests. Thresholds define feature-level drift and a minimum drift ratio triggers CT. Drift reports are uploaded as artifacts for audit and traceability.

6.9 Explainability

SHAP feature importance is stored as an artifact for interpretability. Feature lists, training metadata, and model versions provide transparent evidence for how the model was trained and which inputs drive predictions.

6.10 Sustainability

Training records include emissions estimates (CodeCarbon). Lightweight preprocessing and DVC caching reduce redundant computation. Training is triggered only when necessary to reduce unnecessary compute and energy use.

7. Operational Thresholds, Quality Gates, and Deployment Strategy

7.1 Data Drift Thresholds

Continuous Monitoring applies explicit drift thresholds to decide when to retrain:

- **PSI threshold:** 0.20. A numerical feature is flagged as drifted when PSI exceeds 0.20.
- **KS/Chi2 threshold:** 0.05. A feature is flagged as drifted when the KS or Chi2 p-value is below 0.05.
- **Minimum drift ratio:** 0.30. Continuous Training is triggered only when at least 30% of features are drifted.
- **Fail-on-drift:** Monitoring exits with non-zero status when drift exceeds thresholds to make drift visible and trigger CT.

7.2 Pipeline Quality Checks

Quality checks are enforced at multiple points:

- **Data validation:** schema presence, minimum row counts, datetime parseability, numeric range checks, and non-negative target checks before training.
- **Champion vs challenger gate:** training fails if RMSLE regresses by more than 5% compared with the last accepted model.
- **CI test gate:** lint and unit tests must pass; coverage threshold is enforced in CI.
- **Readiness vs liveness:** /ready returns 503 when the model is unavailable, preventing traffic until the model is loaded.

7.3 Deployment Strategy

Deployment follows a safe, traceable strategy:

- **Immutable image tags:** images are pushed with commit SHA tags (and latest for convenience).
- **Artifact Registry:** images are published to a private registry with pull secrets on the cluster.
- **Rolling updates:** Kubernetes rolling updates limit downtime via maxSurge and maxUnavailable.

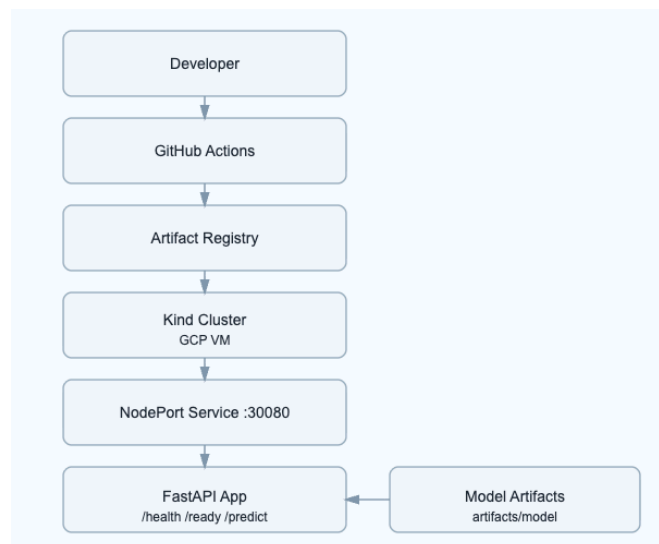
- **Rollout verification:** deployment waits for rollout completion with a timeout; failures trigger rollback when a previous revision exists.
- **Smoke test:** a post-deploy health check against /ready validates service readiness before declaring success.

8. CI/CD and CT/CM Workflows

- **Ingest and Preprocess:** downloads Kaggle data, validates, and produces processed artifacts.
- **Train Model:** trains, logs to MLflow, uploads artifacts and version metadata.
- **Continuous Training:** retrains when bronze data updates, validates data quality.
- **Continuous Monitoring:** runs drift checks and triggers CT if drift is detected.
- **Deploy:** builds and pushes images to Artifact Registry, deploys to Kubernetes with rollout verification.

9. Deployment Architecture

- **Runtime:** Kind Kubernetes cluster on a GCP VM with a self-hosted GitHub Actions runner.
- **Networking:** NodePort service exposes the API to external clients.
- **Registry:** Artifact Registry hosts Docker images; image pull secrets allow the cluster to authenticate.
- **Operational flow:** CI builds and tags images, CD applies manifests, and rollout is verified before smoke testing.



10. Secrets and Access

- Kaggle credentials are stored as GitHub Actions secrets.
- GCP service account key is stored as a GitHub secret and used for Artifact Registry and GCS access.
- Kubernetes pull secret authenticates to Artefact Registry for image pulls.

11. Academic and Technical References

1. Sculley, D. et al. (2015). Hidden Technical Debt in Machine Learning Systems. NIPS.
2. Breck, E. et al. (2017). The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction. IEEE Big Data.
3. Baylor, D. et al. (2017). TFX: A TensorFlow-Based Production-Scale Machine Learning Platform. KDD.
4. Rao, A., and Venkatesh, S. (2015). Data Validation for Machine Learning Pipelines. IEEE Data Engineering.
5. Gama, J., Zliobaite, I., Bifet, A., Pechenizkiy, M., and Bouchachia, A. (2014). A Survey on Concept Drift Adaptation. ACM Computing Surveys.
6. Lundberg, S. M., and Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS.
7. Jain, S., and Neal, R. M. (2015). Structural Causal Models and Explainability. Journal of Machine Learning Research.
8. CodeCarbon: Lacoste, A., Luccioni, A., Schmidt, V., and Dandres, T. (2019). Quantifying Carbon Emissions of Machine Learning. arXiv:1910.09700.
9. DVC: Kuprieiev, V. et al. (2019). Data Version Control. <https://dvc.org>
10. MLflow: Zaharia, M. et al. (2018). Accelerating the Machine Learning Lifecycle with MLflow. <https://mlflow.org>

12. Additional References

- Kaggle Bike Sharing Demand: <https://www.kaggle.com/competitions/bike-sharing-demand>
- FastAPI: <https://fastapi.tiangolo.com/>
- Google Artifact Registry: <https://cloud.google.com/artifact-registry>
- Kubernetes: <https://kubernetes.io/>